

Homework 2

This Notebook will detail Homework 2, which involves a basic capacity expansion model formulation described in [Notebook 3](#)

First, load (or install if necessary) a set of packages you'll need for this assignment...

```
In [1]: # Uncomment and run this first line if you need to install or update packages
import Pkg; Pkg.add("JuMP"); Pkg.add("HiGHS"); Pkg.add("DataFrames"); Pkg.add("CSV")
using JuMP
using HiGHS
using DataFrames
using CSV, Statistics
```

Question 1 - Build the basic thermal generation expansion model

Using the example model in [Notebook 3](#) as your guide, input the code to create a basic thermal generator capacity expansion model, including [downloading the data for Notebook 3 here](#) and loading the appropriate csv files.

```
In [45]: # Initialize data
generators = DataFrame(CSV.File("expansion_data/generators_for_expansion.csv"))
G = generators.G[1:(size(generators,1)-2)]
demand = DataFrame(CSV.File("expansion_data/demand_for_expansion.csv"))
H = demand.Hour
NSECost = 9000

# Model formulation
exp_model = Model(HiGHS.Optimizer)

@variables(exp_model, begin
    CAP[g in G] >= 0           # Gen cap built (MW)
    GEN[g in G, h in H] >= 0  # Gen/hr (MWh)
    NSE[h in H] >= 0          # Non-served energy/ hr (MWh)
end)

@constraint(exp_model, [h in H],
    sum(GEN[g,h] for g in G) + NSE[h] == demand.Demand[h]
)

@constraint(exp_model, [g in G, h in H],
    GEN[g,h] <= CAP[g]
)

@objective(exp_model, Min,
    sum(generators[generators.G .== g, :FixedCost][1] * CAP[g] +
        sum(generators[generators.G .== g, :VarCost][1] * GEN[g,h] for h in H)
        for g in G)
    + sum(NSECost * NSE[h] for h in H)
)

optimize!(exp_model)
```

```

# Total generation by resource
generation = Dict{g => sum(value.(GEN[g, :])) for g in G}

# Installed capacity
capacity = Dict{g => value(CAP[g]) for g in G}

# Non-served energy
NSE_MWh = sum(value.(NSE))
NSE_MW = maximum(value.(NSE))

# Output results
println("Optimal capacity (MW): ", capacity)
println("Annual generation (MWh): ", generation)
println("Total NSE (MWh): ", NSE_MWh)
println("Max hourly NSE (MW): ", NSE_MW)

```

Running HiGHS 1.13.1 (git hash: 1d267d97c): Copyright (c) 2026 under Apache 2.0 license terms

Using BLAS: blastrampoline-5

LP has 43800 rows; 43804 cols; 113880 nonzeros

Coefficient ranges:

```

Matrix [1e+00, 1e+00]
Cost    [2e+01, 6e+05]
Bound   [0e+00, 0e+00]
RHS     [1e+03, 5e+03]

```

Presolving model

43800 rows, 43804 cols, 113880 nonzeros 0s

Dependent equations search running on 8760 equations with time limit of 1000.00s

Dependent equations search removed 0 rows and 0 nonzeros in 0.00s (limit = 1000.00s)

43800 rows, 43804 cols, 113880 nonzeros 0s

Presolve reductions: rows 43800(-0); columns 43804(-0); nonzeros 113880(-0) - Not reduced

Problem not reduced by presolve: solving the LP

Using dual simplex solver

```

Iteration      Objective      Infeasibilities num(sum)
      0         0.0000000000e+00 Pr: 8760(2.25679e+07) 0.3s
    27493      9.9087397880e+08 Pr: 0(0) 0.8s

```

Model status : Optimal

Simplex iterations: 27493

Objective value : 9.9087397880e+08

P-D objective error : 3.3144637923e-14

HiGHS run time : 0.85

Optimal capacity (MW): Dict{String7, Float64}("CT" => 1290.0, "Coal" => 0.0, "CCGT" => 3328.0, "Geo" => 0.0)

Annual generation (MWh): Dict{String7, Float64}("CT" => 427894.0, "Coal" => -0.0, "CCGT" => 2.2139366e7, "Geo" => -0.0)

Total NSE (MWh): 637.0

Max hourly NSE (MW): 195.0

In []:

Question 2: Analytical solution

A. Using the data provided above, sort the demand data from highest to lowest hours to create a load duration curve and save this as a vector/array/DataFrame of your choice.

In [46]: LDC_df = sort(demand, :Demand, rev = true)

Out[46]: 8760×2 DataFrame

8735 rows omitted

Row	Hour	Demand
	Int64	Int64
1	5399	4813
2	5400	4784
3	5398	4745
4	5423	4677
5	5401	4670
6	5424	4645
7	5397	4629
8	5422	4618
9	5425	4555
10	5421	4514
11	5256	4498
12	5255	4496
13	5328	4496
⋮	⋮	⋮
8749	1332	1356
8750	1019	1350
8751	1522	1349
8752	1354	1344
8753	2530	1344
8754	971	1337
8755	2026	1337
8756	706	1333
8757	1331	1326
8758	1187	1321
8759	1018	1319
8760	1186	1306



B. Now using the cost data provided in `'/generators_for_expansion.csv'` and the load duration curve above, use the formulas provided in Lecture to determine an analytical solution to the optimal thermal generation expansion decisions (e.g. solve it algebraically rather than use an optimization solver to find the solution).

Report the optimal capacity of each generation source and compare to the solution from the optimization model above.

Show your work in cells below, using Julia to perform calculations. Explain your steps using inline code comments (e.g. `# Comment`) or by interspersing Markdown cells.

Tip: round your solutions for the crossover hour between each technology to the nearest integer (as we have discrete hours in the time series).

```
In [54]: # Fixed and Var cost calc for thermal generators from file
gen = generators[1:(size(generators,1)-2), :] # drop wind + solar
gen.FixedCost = gen.Capex .* gen.CRF .+ gen.FixedOM
gen.VarCost    = gen.VarOM .+ gen.HeatRate .* gen.FuelCost

# Extract fixed and var cost
F_CCGT = gen.FixedCost[gen.G .== "CCGT"][1]
V_CCGT = gen.VarCost[gen.G .== "CCGT"][1]

F_CT   = gen.FixedCost[gen.G .== "CT"][1]
V_CT   = gen.VarCost[gen.G .== "CT"][1]

# NSE fixed and var costs
F_NSE = 0.0
V_NSE = NSECost

# Screening curve break even CF
CF_CCGT_CT = (F_CT - F_CCGT) / (8760 * (V_CCGT - V_CT))
CF_CT_NSE  = (F_NSE - F_CT) / (8760 * (V_CT - V_NSE))

println("Break even CF CCGT = CT: ", CF_CCGT_CT)
println("Break even CF CT = NSE:  ", CF_CT_NSE)

# Compute CF for each cap
sorted = LDC_df.Demand # Load Duration Curve Load in
peak = maximum(sorted) # max value from LDC_df
unique_caps = sort(unique(sorted)) #reduce demand sorting to exclusively unique values in descending order
CF = [count(sorted .> c) / 8760 for c in unique_caps]

function cap_at_CF(target) # takes break even values and finds the capacity LDC
    idx = argmin(abs.(CF .- target)) # Find closest absolute difference between CF and target
    return unique_caps[idx]
end

# Breakpoint capacities
C1 = cap_at_CF(CF_CCGT_CT) # CCGT = CT
C2 = cap_at_CF(CF_CT_NSE)  # CT = NSE

# Analytical optimal capacities
CAP_CCGT = C1
CAP_CT   = max(C2 - C1, 0) # point where CT kicks in is max between C1 and C2
CAP_NSE  = max(peak - C2, 0) # similar logic, just with peak and c2 instead

# print for checking and comparison
println("\n Analytical Optimal Capacities")
println("CCGT = ", round(CAP_CCGT, digits=1), " MW")
println("CT   = ", round(CAP_CT,   digits=1), " MW")
println("NSE  = ", round(CAP_NSE,  digits=1), " MW")

# Load LP model results for printing
LP_CCGT = capacity["CCGT"]
LP_CT   = capacity["CT"]
LP_NSE  = NSE_MW # max hourly NSE from earlier model
```

```
println("\nLP Model Capacities Comparison")
println("CCGT = ", LP_CCGT, " MW")
println("CT   = ", LP_CT,   " MW")
println("NSE   = ", LP_NSE,  " MW")
```

Break even CF CCGT = CT: 0.13749365804160327

Break even CF CT = NSE: 0.0008015671192905423

Analytical Optimal Capacities

CCGT = 3328.0 MW

CT = 1290.0 MW

NSE = 195.0 MW

LP Model Capacities Comparison

CCGT = 3328.0 MW

CT = 1290.0 MW

NSE = 195.0 MW

C. Now change the fuel cost of natural gas to \$8.00/MMBtu, recalculate the variable cost of CCGTs and CTs, and solve again for the optimal generation capacity mix. Describe what changes in your capacity results and what doesn't, and provide an explanation.

```
In [55]: # Initialization
gen_new = generators[1:(size(generators,1)-2), :]
gen_new.FixedCost = gen_new.Capex .* gen_new.CRF .+ gen_new.FixedOM

# Update fuel costs to $8/MMBtu in gas turbines
for g in ["CCGT", "CT"]
    idx = findfirst(==(g), gen_new.G)
    gen_new.FuelCost[idx] = 8.0
end

# Recompute var cost fixed cost are the same as before
gen_new.VarCost = gen_new.VarOM .+ gen_new.HeatRate .* gen_new.FuelCost

# Extract updated costs
F_CCGT_new = gen_new.FixedCost[gen_new.G .== "CCGT"][1]
V_CCGT_new = gen_new.VarCost[gen_new.G .== "CCGT"][1]

F_CT_new = gen_new.FixedCost[gen_new.G .== "CT"][1]
V_CT_new = gen_new.VarCost[gen_new.G .== "CT"][1]

# NSE
F_NSE = 0.0
V_NSE = NSECost

# Break even CFs
#CF_CCGT_CT_new = (F_CT_new - F_CCGT_new) / (8760 * (V_CCGT_new - V_CT_new))
#CF_CT_NSE_new = (F_NSE - F_CT_new) / (8760 * (V_CT_new - V_NSE))

CF_CCGT_CT_new = (F_CCGT_new - F_CT_new) / (8760 * (V_CT_new - V_CCGT_new))
CF_CT_NSE_new = (F_CT_new - F_NSE) / (8760 * (V_NSE - V_CT_new))

println("Break even CF CCGT = CT: ", CF_CCGT_CT_new)
println("Break even CF CT = NSE: ", CF_CT_NSE_new)

# Use existing LDC from Part A and same method as B
sorted = LDC_df.Demand
unique_caps = sort(unique(sorted))
CF_of_c = [count(sorted .> c) / 8760 for c in unique_caps]
cap_at_CF(target) = unique_caps[argmin(abs.(CF_of_c .- target))]
```

```

# New breakpoint capacities
C1_new = cap_at_CF(CF_CCGT_CT_new)
C2_new = cap_at_CF(CF_CT_NSE_new)
peak   = maximum(sorted)

# New analytical capacities
CAP_CCGT_new = C1_new
CAP_CT_new   = max(C2_new - C1_new, 0)
CAP_NSE_new  = max(peak - C2_new, 0)

println("\nNew Analytical Optimal Capacities (Fuel = \$8)")
println("CCGT = ", round(CAP_CCGT_new, digits=1), " MW")
println("CT   = ", round(CAP_CT_new,   digits=1), " MW")
println("NSE   = ", round(CAP_NSE_new, digits=1), " MW")

# Compare directly to Part B analytical results
println("\nComparison to Base Case (from Part B)")
println("CCGT: base = ", round(CAP_CCGT, digits=1), " >> new = ", round(CAP_CCGT_new, digits=1))
println("CT:   base = ", round(CAP_CT,   digits=1), " >> new = ", round(CAP_CT_new,   digits=1))
println("NSE:  base = ", round(CAP_NSE,  digits=1), " >> new = ", round(CAP_NSE_new, digits=1))

```

Break even CF CCGT = CT: 0.07576181157394467
 Break even CF CT = NSE: 0.0008050547731165983

New Analytical Optimal Capacities (Fuel = \$8)
 CCGT = 3581.0 MW
 CT = 1037.0 MW
 NSE = 195.0 MW

Comparison to Base Case (from Part B)
 CCGT: base = 3328.0 >> new = 3581.0
 CT: base = 1290.0 >> new = 1037.0
 NSE: base = 195.0 >> new = 195.0

The results of the analytical optimization in 2C show that for the fuel cost of natural gas as 8.00/MMBtu,

the optimal capacity for CCGT increases and CT decreases when compared to the results from B which is 4/MMBtu. The variable costs of CT rise much quicker than CCGT because the heat rate of CT is so much higher than that of CCGT, making the price hike in fuel costs much more prevalent in the variable cost for that generator. This makes it more reasonable to build more CCGT even with its higher fixed costs, since its usage is more justified at lower utilization rates than before. The NSE value does not change between either scenario which makes sense, since that value is determined by the cross over between CT and NSE, which is reliant on fixed costs for CT. Since the CT fixed costs did not change and NSE remained at \$9000/MWh, the optimal value also does not change between part B and C, since that decision is based on whether to build capacity at peaks.



Question 3 - Expansion with renewables

A. Using JuMP/Julia, implement an optimization model based on the formulation for optimal thermal+renewable capacity expansion provided in Section 2 of [Notebook 3](#).

```

In [ ]: # Load in variability data for renewables
variability = DataFrame(CSV.File("expansion_data/wind_solar_for_expansion.csv"))

```

```

G = generators.G # Reset G to include renewables
G_ren = ["Wind", "Solar"]
G_therm = G[1:4] # specific to thermal

# Initialize
exp_model_re = Model(HiGHS.Optimizer)

@variables(exp_model_re, begin
    CAP[g in G] >= 0
    GEN[g in G, h in H] >= 0
    NSE[h in H] >= 0
end)

# Demand balance
@constraint(exp_model_re, [h in H],
    sum(GEN[g,h] for g in G) + NSE[h] == demand.Demand[h]
)

# Capacity constraints thermal and renewable
@constraint(exp_model_re, [g in G_therm, h in H],
    GEN[g,h] <= CAP[g]
)

@constraint(exp_model_re, [g in G_ren, h in H],
    GEN[g,h] <= CAP[g] * variability[h, Symbol(g)] # includes variability
)

# Objective
@objective(exp_model_re, Min,
    sum(generators[generators.G == g, :FixedCost][1] * CAP[g] +
        sum(generators[generators.G == g, :VarCost][1] * GEN[g,h] for h in H)
        for g in G) +
    sum(NSECost * NSE[h] for h in H)
)

# Solve
optimize!(exp_model_re)

# Results
cap_opt = Dict{g => value(CAP[g]) for g in G}
gen_opt = Dict{g => sum(value.(GEN[g,:])) for g in G}
NSE_MWh = sum(value.(NSE))
NSE_MW = maximum(value.(NSE))

println("Optimal capacity (MW): ", cap_opt)
println("Annual generation (MWh): ", gen_opt)
println("Total NSE (MWh): ", NSE_MWh)
println("Max hourly NSE (MW): ", NSE_MW)

```


Using BLAS: blastrampoline-5

LP has 61320 rows; 61326 cols; 161976 nonzeros

Coefficient ranges:

Matrix [5e-05, 1e+00]
 Cost [2e+01, 6e+05]
 Bound [0e+00, 0e+00]
 RHS [1e+03, 5e+03]

Presolving model

56856 rows, 56862 cols, 153048 nonzeros 0s

Dependent equations search running on 8760 equations with time limit of 1000.00s

Dependent equations search removed 0 rows and 0 nonzeros in 0.00s (limit = 1000.00s)

56856 rows, 56862 cols, 153048 nonzeros 0s

Presolve reductions: rows 56856(-4464); columns 56862(-4464); nonzeros 153048(-8928)

Solving the presolved LP

Using dual simplex solver

Iteration	Objective	Infeasibilities	num(sum)
0	0.0000000000e+00	Pr: 8760(6.18517e+06)	0.3s
40370	8.2899893531e+08	Pr: 0(0)	5.1s

Performed postsolve

Solving the original LP from the solution after postsolve

Model status : Optimal

Simplex iterations: 40370

Objective value : 8.2899893531e+08

P-D objective error : 3.1348200823e-14

HiGHS run time : 5.19

Optimal capacity (MW): Dict{String7, Float64}("CT" => 1419.7834328744516, "Coal" => 0.0, "CCGT" => 2422.855352805202, "Wind" => 349.95573041870074, "Geo" => 0.0, "Solar" => 3362.7680693765346)

Annual generation (MWh): Dict{String7, Float64}("CT" => 453979.74168545875, "Coal" => 0.0, "CCGT" => 1.1367656651128398e7, "Wind" => 1.007696972638563e6, "Geo" => 0.0, "Solar" => 9.7381643068072e6)

Total NSE (MWh): 399.3277404673072

Max hourly NSE (MW): 136.5133200973096

B. Solve the model to determine the optimal capacity when wind and solar are available resources and extract results for generation and capacity.

```
In [67]: # Results DataFrame
results_re = DataFrame(
    Resource      = G,
    CAP_MW        = [round(value(CAP[g]), digits=1) for g in G],
    CAP_Pct       = [round(value(CAP[g]) / sum(value(CAP[g]) for g in G) * 100, digits=1) for g in G],
    GEN_GWh       = [round(sum(value.(GEN[g,:])) / 1000, digits=1) for g in G],
    GEN_Pct       = [round(sum(value.(GEN[g,:])) / sum(value.(GEN[g,h] for g in G, h in H)) * 100,
)

# Append NSE row
push!(results_re, ("NSE", round(NSE_MW, digits=1), round(NSE_MW / sum(value(CAP[g]) for g in G) * 100, digits=1),
                  round(NSE_MWh / 1000, digits=3), round(NSE_MWh / sum(value.(GEN[g,h] for g in G, h in H)) * 100, digits=1))

results_re
```


Out[67]: 7×5 DataFrame

Row	Resource	CAP_MW	CAP_Pct	GEN_GWh	GEN_Pct
	String7	Float64	Float64	Float64	Float64
1	Geo	0.0	0.0	0.0	0.0
2	Coal	0.0	0.0	0.0	0.0
3	CCGT	2422.9	32.1	11367.7	50.4
4	CT	1419.8	18.8	454.0	2.0
5	Wind	350.0	4.6	1007.7	4.5
6	Solar	3362.8	44.5	9738.2	43.2
7	NSE	136.5	1.8	0.399	0.002

C. What happens to the total firm generation and maximum MW of non-served energy? What does this imply about the capacity value of solar and/or wind built in the optimal capacity mix?

The firm generation total sees the total installed capacity go up. CCGT decreased in installed capacity from the original scenario with no renewables since the wind and solar deployment take up a decent chunk of its role as baseload generation. CT increases because while wind and solar serve as partial baseload generators, the need for peaking and quickly dispatchable generation is needed, so CT capacity is expanded to meet that need. CT capacity also meets demand when solar is lower but demand is still high. The sum total capacity is significantly larger than in the non renewable case because the fact that renewables are variable forces overbuilding to ensure demand is met. The maximum MW of non served energy, 136.5 MW, is a decrease from before because of the zero variable cost of wind and solar and the increased presence of peaker generation capacity. All of this serves to imply that solar and wind have limited capacity value, they help serve a large part of the baseload when active and decrease NSE in this scenario slightly, but they require massive over deployment to ensure reliability. Firm peaking capabilities are still required to serve load since this scenario has no storage options.

Question 4: Brownfield Expansion Model

A. Now implement an optimization model based on the formulation for optimal "brownfield" thermal+renewable capacity expansion (e.g. with existing generators) provided in Section 3 of [Notebook 3](#).

Use the following data for fixed and variable costs of existing gas capacity. Note: unlike in the formulation in Notebook 3, there is no existing renewable capacity here to consider (only thermal).

```
In [ ]: # Load new generator options
generators_bf = DataFrame(CSV.File("expansion_data/generators_for_expansion.csv"))
# Add parameters for existing CCGTs, with the set index "Old"
push!(generators_bf, ["Old_CCGT" "Existing CCGT" 0 40000 5 7.5 4 0 0 0 0 40000 30])
# Add parameters for existing CTs, with the set index "Old"
push!(generators_bf, ["Old_CT" "Existing CT" 0 30000 11 11.0 4 0 0 0 0 30000 55])

# Set installed capacity for existing CCGTs:
ExistingCap_CCGT = 1260 # Approximate actual existing capacity in SDGE
```

```

ExistingCap_CT = 925 # Approximate actual existing capacity in SDGE
# Add new column to generators Data Frame
generators_bf[!,:ExistingCap] = [0,0,0,0,0,0, ExistingCap_CCGT, ExistingCap_CT];

# Initialize
G_new = generators_bf.G[1:6]      # new build options (thermal + renewables)
G_old = ["Old_CC", "Old_CT"]      # existing generators
G_all_bf = generators_bf.G        # all generators
G_therm_new = generators_bf.G[1:4] # new thermal only
G_ren = ["Wind", "Solar"]         # renewables

# Build brownfield model
bf_model = Model(HiGHS.Optimizer)

@variables(bf_model, begin
    CAP[g in G_new] >= 0          # new build cap(MW)
    RET[g in G_old] >= 0          # retirement (MW)
    GEN[g in G_all_bf, h in H] >= 0 # generation all resources
    NSE[h in H] >= 0
end)

# Demand balance
@constraint(bf_model, [h in H],
    sum(GEN[g,h] for g in G_all_bf) + NSE[h] == demand.Demand[h]
)

# Capacity constraints (new thermal)
@constraint(bf_model, [g in G_therm_new, h in H],
    GEN[g,h] <= CAP[g]
)

# Capacity constraints (new renewables)
@constraint(bf_model, [g in G_ren, h in H],
    GEN[g,h] <= CAP[g] * variability[h, Symbol(g)]
)

# Capacity constraints (existing)
@constraint(bf_model, [g in G_old, h in H],
    GEN[g,h] <= generators_bf[generators_bf.G .== g, :ExistingCap][1] - RET[g]
)

# retirement cap
@constraint(bf_model, [g in G_old],
    RET[g] <= generators_bf[generators_bf.G .== g, :ExistingCap][1]
)

# Objective
@objective(bf_model, Min,
    sum(generators_bf[generators_bf.G .== g, :FixedCost][1] * CAP[g] for g in G_new) +
    sum(generators_bf[generators_bf.G .== g, :FixedCost][1] *
        (generators_bf[generators_bf.G .== g, :ExistingCap][1] - RET[g]) for g in G_old) +
    sum(generators_bf[generators_bf.G .== g, :VarCost][1] * GEN[g,h]
        for g in G_all_bf, h in H) +
    sum(NSECost * NSE[h] for h in H)
)

optimize!(bf_model)

```

Running HiGHS 1.13.1 (git hash: 1d267d97c): Copyright (c) 2026 under Apache 2.0 license terms
Using BLAS: blastrampoline-5
LP has 78842 rows; 78848 cols; 214538 nonzeros
Coefficient ranges:
Matrix [5e-05, 1e+00]
Cost [2e+01, 6e+05]
Bound [0e+00, 0e+00]
RHS [9e+02, 5e+03]

Presolving model

74376 rows, 74384 cols, 205608 nonzeros 0s

Dependent equations search running on 8760 equations with time limit of 1000.00s

Dependent equations search removed 0 rows and 0 nonzeros in 0.00s (limit = 1000.00s)

74376 rows, 74384 cols, 205608 nonzeros 0s

Presolve reductions: rows 74376(-4466); columns 74384(-4464); nonzeros 205608(-8930)

Solving the presolved LP

Using dual simplex solver

Iteration	Objective	Infeasibilities num(sum)
0	0.0000000000e+00	Ph1: 0(0) 0.4s
35228	6.5098622812e+08	Pr: 17816(6.06625e+06); Du: 0(6.45801e-08) 5.8s
39906	7.1909698392e+08	Pr: 12142(3.32355e+06); Du: 0(6.45801e-08) 11.4s
42155	7.4034239058e+08	Pr: 12171(3.78714e+06); Du: 0(6.45801e-08) 16.5s
43616	7.4451886574e+08	Pr: 11029(1.33753e+06); Du: 0(6.45801e-08) 21.7s
45666	7.5403253063e+08	Pr: 1260(50145.8); Du: 0(6.45801e-08) 27.5s
46784	7.5494110973e+08	Pr: 0(0); Du: 0(6.18883e-12) 30.9s

Performed postsolve

Solving the original LP from the solution after postsolve

Model status : Optimal

Simplex iterations: 46784

Objective value : 7.5494110973e+08

P-D objective error : 3.1817941187e-14

HiGHS run time : 31.25

B. Solve the model to determine the optimal capacity when with existing generators and extract results for generation and capacity (including retirements).

```
In [ ]: # Extract results
cap_new = Dict{g => value(CAP[g]) for g in G_new}
ret_old = Dict{g => value(RET[g]) for g in G_old}
gen_all = Dict{g => sum(value.(GEN[g,:])) for g in G_all_bf}
NSE_MWh_bf = sum(value.(NSE))
NSE_MW_bf = maximum(value.(NSE))

# New build results
results_new = DataFrame(
    Resource = G_new,
    Type = fill("New Build", length(G_new)),
    CAP_MW = [round(cap_new[g], digits=1) for g in G_new],
    GEN_GWh = [round(gen_all[g] / 1000, digits=1) for g in G_new]
)

# Existing capacity results
results_old = DataFrame(
    Resource = G_old,
    Type = fill("Existing", length(G_old)),
    CAP_MW = [round(generators_bf[generators_bf.G == g, :ExistingCap][1] - value(RET[g]), digits=1) for g in G_old],
    GEN_GWh = [round(gen_all[g] / 1000, digits=1) for g in G_old]
)

# NSE row
```

```

results_nse = DataFrame(
    Resource = ["NSE"],
    Type      = ["Non Served"],
    CAP_MW    = [round(NSE_MW_bf, digits=1)],
    GEN_GWh   = [round(NSE_MWh_bf / 1000, digits=3)]
)

# Combined table
results_bf = vcat(results_new, results_old, results_nse)

results_bf

```

Out[]: 9×4 DataFrame

	Row	Resource	Type	CAP_MW	GEN_GWh
		String	String	Float64	Float64
	1	Geo	New Build	0.0	0.0
	2	Coal	New Build	0.0	0.0
	3	CCGT	New Build	1417.3	8545.0
	4	CT	New Build	238.6	105.9
	5	Wind	New Build	385.0	1109.3
	6	Solar	New Build	3357.6	9720.2
	7	Old_CC	Existing	1260.0	2973.4
	8	Old_CT	Existing	925.0	113.6
	9	NSE	Non-Served	125.1	0.366



```

In [ ]: # Retirements
println("\nRetirement Decisions")
for g in G_old
    existing = generators_bf[generators_bf.G .== g, :ExistingCap][1]
    retired   = round(value(RET[g]), digits=1)
    retained  = round(existing - value(RET[g]), digits=1)
    println("$g: Existing = $existing MW , Retired = $retired MW , Retained = $retained MW")
end

```

Retirement Decisions

Old_CC: Existing = 1260 MW , Retired = 0.0 MW , Retained = 1260.0 MW

Old_CT: Existing = 925 MW , Retired = 0.0 MW , Retained = 925.0 MW

In []: